

Memory Management & Garbage Collection

Level: Advanced / Senior Developer

Topics: 07-11

Goal: JavaScript runtime কীভাবে memory allocate করে, objects কোথায় থাকে, Garbage Collector কীভাবে unused objects reclaim করে, memory leak কেন হয় এবং Chrome DevTools দিয়ে leak কীভাবে investigate করতে হয় তা বোঝা।

1. Memory Management কী?

JavaScript program চলার সময় বিভিন্ন data memory-তে রাখতে হয়।

যেমন:

```
const name = "Shariar";

const user = {
  name: "Shariar",
  age: 31
};

const numbers = [10, 20, 30];
```

Runtime-কে এই data রাখার জন্য memory ব্যবহার করতে হয়।

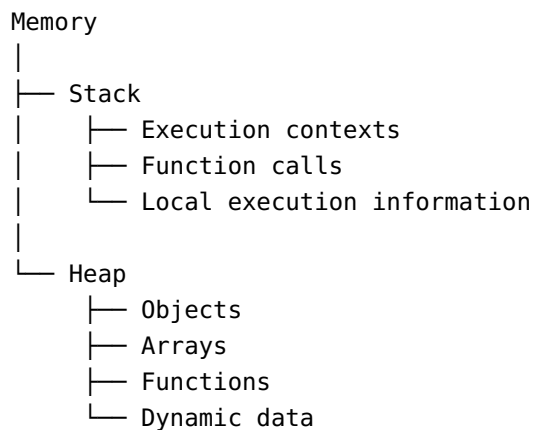
Simplified model:

```
JavaScript Program
  ↓
Memory Allocation
  ↓
Objects / Arrays / Functions
  ↓
Objects become unreachable
  ↓
Garbage Collector
  ↓
Memory Reclaimed
```

2. Stack বনাম Heap

JavaScript memory বোঝার সময় Stack এবং Heap-এর ধারণা আসে।

Simplified mental model:



তবে এটাকে JavaScript specification-এর exact memory layout ভাববে না।

ECMAScript specification সাধারণত “Stack” এবং “Heap” এভাবে define করে না। এগুলো engine/runtime বোঝার জন্য useful conceptual model।

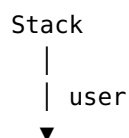
3. JavaScript Heap

Heap হলো runtime-এর dynamic memory area যেখানে objects এবং অন্যান্য dynamically allocated data রাখা হয়।

যেমন:

```
const user = {  
  name: "A",  
  age: 25  
};
```

Conceptually:



Heap

```
name: "A"  
age: 25
```

আর:

```
const users = [  
  { name: "A" },  
  { name: "B" }  
];
```

এর মতো complex objects-এর জন্য heap allocation গুরুত্বপূর্ণ।

4. Heap কেন গুরুত্বপূর্ণ?

কারণ heap-এর memory unlimited নয়।

যদি application ক্রমাগত নতুন objects তৈরি করে:

```
function createData() {  
  return {  
    value: new Array(10000).fill("data")  
  };  
}
```

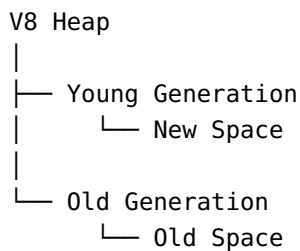
এবং সেই objects-এর reference কোথাও ধরে রাখা হয়, তাহলে memory usage বাড়তে থাকবে।

```
New Objects  
  ↓  
Heap usage ↑  
  ↓  
GC cannot reclaim reachable objects  
  ↓  
Memory pressure  
  ↓  
Performance problems  
  ↓  
Possible crash
```

5. V8 Heap-এর Generational Model

V8 garbage collection-এর জন্য heap-কে বিভিন্ন logical areas/spaces-এ ভাগ করে।

সবচেয়ে গুরুত্বপূর্ণ concept:



6. New Space / Young Generation

নতুন তৈরি হওয়া objects সাধারণত **young generation**-এর memory area-তে শুরু করে।

Simplified:

```
const user = {  
  name: "A"  
};
```

Conceptually:

```
graph TD; New_Object[New Object] --> Young_Generation[Young Generation];
```

The diagram shows 'New Object' with a downward arrow pointing to 'Young Generation'.

অনেক JavaScript object খুব অল্প সময় বেঁচে থাকে।

যেমন:

```
function calculate() {  
  const temp = {  
    value: 100  
  };  
}
```

```
    return temp.value * 2;
}
```

`temp` পরে unreachable হয়ে যেতে পারে।

তাই young generation-এর objects frequently collect করা লাভজনক।

7. Old Space / Old Generation

যে objects দীর্ঘ সময় survive করে তারা eventually **old generation**-এ promote হতে পারে।

Simplified:

```
Object created
  ↓
New Space
  ↓
Survives GC
  ↓
Survives again
  ↓
Promotion
  ↓
Old Space
```

এটা exact promotion rule-এর complete description নয়; V8-এর actual GC heuristics আরও sophisticated।

8. Generational Garbage Collection

Generational GC-এর মূল ধারণা:

Most objects are short-lived.

অর্থাৎ:

```
অনেক objects
  ↓
দ্রুত তৈরি
  ↓
দ্রুত unreachable
```

↓
দ্রুত garbage collect

আর comparatively কম objects:

Long-lived objects
↓
Old generation

এই observation ব্যবহার করেই generational GC efficient হতে পারে।

9. Minor GC

Young generation-এর garbage collection-কে broadly **Minor GC** বলা হয়।

Mental model:

New Space
↓
Minor GC
↓
Dead objects → Reclaim
Live objects → Survive / Promote

কারণ young generation ছোট এবং short-lived objects বেশি থাকে, সেখানে frequent collection করা efficient হতে পারে।

10. Major GC

Old generation-এর garbage collection সাধারণত বেশি expensive হতে পারে।

এখানে larger amount of long-lived objects থাকতে পারে।

Simplified:

Old Space
↓
Major / Full GC
↓
Find unreachable objects

↓
Reclaim memory

Modern V8-এর GC architecture অনেক বেশি complex; “Major GC” এবং “Full GC” সবসময় perfectly interchangeable terms নয়।

তাই senior-level understanding-এর জন্য মূল idea হলো:

Young-generation collection এবং old-generation collection-এর cost/behavior আলাদা।

11. Mark-and-Sweep

Garbage Collection-এর সবচেয়ে important conceptual algorithm:

Mark-and-Sweep

এটার basic idea:

```
GC starts
  ↓
Find reachable objects
  ↓
MARK them
  ↓
Unmarked objects
  ↓
SWEEP / reclaim
```

12. Reachability

Garbage Collector-এর জন্য সবচেয়ে গুরুত্বপূর্ণ concept হলো:

Reachability

কোন object যদি program-এর active references থেকে reachable হয়, তাহলে সেটা garbage নয়।

ধরো:

```
const user = {  
  name: "A"  
};
```

`user` variable object-টাকে reference করছে।

```
Root  
↓  
user  
↓  
Object
```

Object reachable।

13. GC Roots

Garbage Collector কোথা থেকে reachability শুরু করবে?

এগুলোকে broadly **GC roots** বলা হয়।

Conceptually roots-এর মধ্যে থাকতে পারে:

- active execution contexts
- global references
- active handles/references
- অন্যান্য runtime roots

Simplified:

```
GC Roots  
|  
├─ Global references  
├─ Active stack references  
└─ Runtime references  
    |  
    ▼  
  Reachable Objects
```

যে object roots থেকে reachable নয়, সেটি eventually reclaim করা সম্ভব।

14. Mark Phase

ধরো:

```
const user = {  
  name: "A",  
  profile: {  
    age: 25  
  }  
};
```

GC conceptualভাবে:

```
Root  
↓  
user  
↓  
User Object  
↓  
Profile Object
```

এই objects reachable।

GC এগুলো mark করবে।

```
[MARKED] user  
[MARKED] user.profile
```

15. Sweep Phase

এখন ধরো heap-এ আরেকটি object আছে:

```
Root  
|  
└─ user → Object A  
  
Object B
```

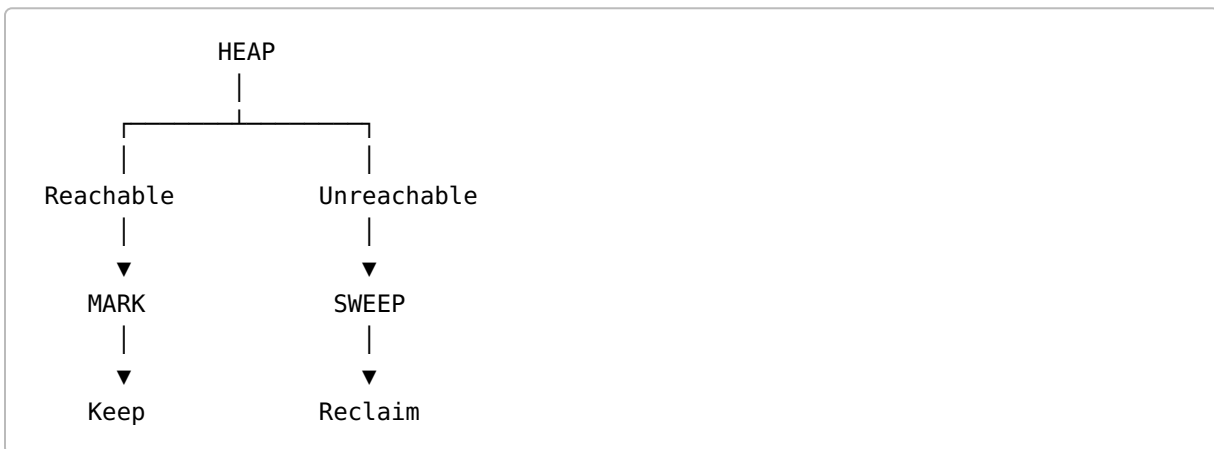
Object B কোনো root থেকে reachable নয়।

তাহলে:

```
Object A → reachable → keep  
Object B → unreachable → reclaim
```

এই reclaim process-এর conceptual অংশ হলো **sweep** ।

16. Mark-and-Sweep Mental Model



17. Circular References

Modern garbage collectors circular reference handle করতে পারে ।

উদাহরণ:

```
let a = {};  
let b = {};  
  
a.other = b;  
b.other = a;
```

এখানে:

```
a → b
↑   ↓
└───┘
```

দুটো object একে অপরকে reference করছে।

কিন্তু পরে:

```
a = null;
b = null;
```

এখন external root থেকে আর reach করা যায় না।

```
GC Root
  |
  x
  |
a ↔ b
```

তাই cycle থাকা সত্ত্বেও objects garbage হতে পারে।

গুরুত্বপূর্ণ

Garbage Collection reference count-এর simple model নয়।

Reachability-based tracing GC circular references handle করতে পারে।

18. Memory Leak কী?

Memory leak হলো এমন পরিস্থিতি যেখানে application-এর আর প্রয়োজন নেই এমন memory references ধরে রাখে, ফলে Garbage Collector memory reclaim করতে পারে না।

সবচেয়ে important distinction:

```
Unused object
  +
Still reachable
  ↓
GC cannot reclaim
```

↓
Memory leak

অর্থাৎ:

Object ব্যবহার করা হচ্ছে না মানেই সেটা garbage নয়।

যদি কোনো live reference থাকে, GC সেটাকে reachable মনে করতে পারে।

19. Memory Leak Example — Global Reference

```
const cache = [];  
  
function processData(data) {  
  cache.push(data);  
}
```

যদি:

```
processData(largeObject1);  
processData(largeObject2);  
processData(largeObject3);
```

এভাবে data জমতেই থাকে:

```
Global cache  
  ↓  
largeObject1  
largeObject2  
largeObject3  
largeObject4  
...
```

`cache` global এবং reachable।

তাই objects garbage collect করা যাবে না যতক্ষণ references রাখা আছে।

20. Accidental Global Variables

Non-strict legacy JavaScript code-এ accidental global তৈরি হতে পারে।

উদাহরণ:

```
function saveData(data) {  
  cachedData = data;  
}
```

যদি `cachedData` properly declared না হয়, কিছু environments-এ এটি global property তৈরি করতে পারে।

Strict mode:

```
"use strict";  
  
function saveData(data) {  
  cachedData = data;  
}
```

এখানে `ReferenceError` হতে পারে।

Best practice

সবসময় declaration ব্যবহার করে:

```
const cachedData = data;
```

অথবা প্রয়োজন অনুযায়ী:

```
let cachedData = data;
```

21. Event Listener Memory Leak

Browser application-এ event listener একটি common source of accidental retention।

ধরো:

```
const button = document.querySelector("#button");

function handleClick() {
  console.log("clicked");
}

button.addEventListener("click", handleClick);
```

যদি component/DOM lifecycle শেষ হওয়ার পর listener এবং related references properly cleanup না হয়, unnecessary objects retained থাকতে পারে।

22. Event Listener Cleanup

যদি listener আর প্রয়োজন না হয়:

```
button.removeEventListener("click", handleClick);
```

Important:

```
button.addEventListener("click", handleClick);
```

এর জন্য একই function reference দরকার:

```
button.removeEventListener("click", handleClick);
```

এইটা:

```
button.removeEventListener("click", () => {
  console.log("clicked");
});
```

আগের listener remove করবে না, কারণ এটা নতুন function object।

23. React-এর ক্ষেত্রে Event Listener

React application-এ manually event listener attach করলে cleanup করতে হবে।

```
useEffect(() => {
  function handleResize() {
    console.log(window.innerWidth);
  }

  window.addEventListener("resize", handleResize);

  return () => {
    window.removeEventListener("resize", handleResize);
  };
}, []);
```

Mental model:

```
Effect starts
↓
Add listener
↓
Component lifecycle ends
↓
Cleanup
↓
Remove listener
```

24. Detached DOM Nodes

একটি DOM node document থেকে remove হয়ে গেছে, কিন্তু JavaScript-এর কোনো reference এখনও সেটাকে ধরে রাখছে।

উদাহরণ:

```
const button = document.querySelector("#button");

const cache = [];

cache.push(button);

button.remove();
```

DOM থেকে button remove হয়েছে।

কিন্তু:

```
cache
↓
button
```

reference এখনও আছে।

তাই object/node-এর memory immediately reclaim করা সম্ভব নাও হতে পারে।

এ ধরনের retained DOM node-কে **detached DOM** বলা হয়।

25. Detached DOM Mental Model

```
Document
↓
DOM Node
```

Node remove:

```
Document
X
|
└─ Node removed
```

কিন্তু:

```
JavaScript cache
↓
Node
```

তাই:

```
DOM detached
+
JS reference exists
↓
Node retained
```

26. Closure Memory Leak

Closure নিজে memory leak নয়।

এটা খুব important।

Closure-এর কারণে outer scope-এর variables reachable থাকতে পারে।

উদাহরণ:

```
function createHandler() {  
  const largeData = new Array(1_000_000).fill("data");  
  
  return function handler() {  
    console.log(largeData.length);  
  };  
}  
  
const handler = createHandler();
```

`handler` function `largeData` -এর সাথে lexical relationship ধরে রাখতে পারে।

যতক্ষণ `handler` reachable:

```
handler  
  ↓  
closure environment  
  ↓  
largeData
```

ততক্ষণ `largeData` retained হতে পারে।

27. Closure Leak কখন হয়?

Closure ব্যবহার করলেই leak নয়।

Leak-like retention হতে পারে যখন:

```
Long-lived callback  
  ↓
```

Closure
↓
Large object

এবং callback-এর lifetime unnecessarily long হয়।

যেমন:

```
const handlers = [];  
  
function register() {  
  const hugeData = new Array(1_000_000).fill("x");  
  
  handlers.push(() => {  
    console.log(hugeData.length);  
  });  
}
```

প্রতিবার `register()` call করলে closure-এর মাধ্যমে `hugeData` retained হতে পারে।

28. Closure বনাম Memory Leak

মনে রাখবে:

Closure ≠ Memory Leak

বরং:

Closure
↓
Can retain references
↓
If retained unnecessarily
↓
Potential memory leak

Closures JavaScript-এর fundamental feature এবং অনেক useful pattern-এর foundation।

29. Common Memory Leak Patterns

Pattern	সমস্যা
Growing global array	References never released
Unbounded cache	Memory continuously grows
Event listeners	Listener/closure retained
Timers	Callback references retained
Detached DOM	JS reference keeps node alive
Long-lived closures	Captured data retained
Large objects in module scope	Long lifetime
Subscription cleanup missing	Objects remain reachable

30. Timer-related Retention

উদাহরণ:

```
const hugeData = new Array(1_000_000).fill("x");

const timer = setInterval(() => {
  console.log(hugeData.length);
}, 1000);
```

যতক্ষণ interval active:

```
Timer
↓
Callback
↓
Closure
↓
hugeData
```

`clearInterval()` না করলে এই chain unnecessarily long-lived হতে পারে।

Cleanup:

```
clearInterval(timer);
```

31. Unbounded Cache

Caching useful:

```
const cache = new Map();  
  
cache.set(key, value);
```

কিন্তু cache-এর কোনো limit না থাকলে:

```
Cache  
↓  
Entry 1  
Entry 2  
Entry 3  
...  
Entry 1,000,000
```

Memory continuously grow করতে পারে।

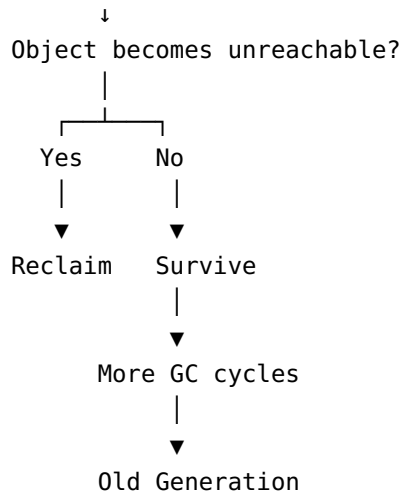
Better architecture:

```
Cache  
├─ max size  
├─ expiration  
├─ eviction  
└─ invalidation
```

32. Garbage Collection-এর Basic Lifecycle

একটি simplified lifecycle:

```
Object created  
↓  
Young Generation
```



33. Minor বনাম Major GC

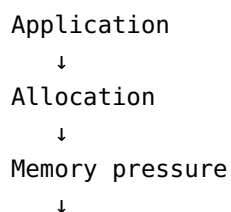
বিষয়	Minor GC	Major/Old-generation GC
প্রধান target	Young objects	Older objects
সাধারণত	More frequent	Less frequent
Scope	Smaller	Larger
Cost	Usually lower	Potentially higher
Main idea	Short-lived objects	Long-lived objects

Exact V8 behavior implementation/version অনুযায়ী পরিবর্তিত হতে পারে।

34. GC Pause

Garbage Collection-এর কিছু phase application execution-এর সাথে compete করতে পারে।

যদি GC অনেক কাজ করে:



GC work
↓
CPU time

ফলে application responsiveness/performance প্রভাবিত হতে পারে।

Modern V8 incremental/concurrent/parallel techniques ব্যবহার করে pause এবং GC overhead কমানোর চেষ্টা করে।

35. Garbage Collection মানেই Free

এটা ভুল:

“JavaScript automatic GC করে, তাই memory management নিয়ে চিন্তা করতে হয় না।”

সঠিক ধারণা:

Automatic GC
↓
Automatic reclamation of unreachable memory

কিন্তু:

Memory Leak
↓
Object still reachable
↓
GC cannot reclaim

তাই developer-এর responsibility:

unnecessary references ধরে না রাখা।

36. Chrome DevTools — Memory Tab

Browser application-এর memory investigate করার জন্য Chrome DevTools-এর **Memory** panel গুরুত্বপূর্ণ।

সাধারণত DevTools খুলে:

Chrome DevTools



Memory

এখানে বিভিন্ন profiling options পাওয়া যায়।

সবচেয়ে গুরুত্বপূর্ণ:

- Heap Snapshot
- Allocation instrumentation/timeline
- Allocation sampling

37. Heap Snapshot

Heap Snapshot হলো নির্দিষ্ট মুহূর্তে JavaScript heap-এর memory/object graph-এর snapshot।

এটা দিয়ে investigate করা যায়:

- কোন objects আছে
- কোন object বেশি memory নিচ্ছে
- কোন constructor বেশি instances তৈরি করেছে
- object retained কেন হচ্ছে
- references কোথা থেকে আসছে

38. Heap Snapshot-এর Mental Model

Application Running



Take Snapshot



Inspect Objects



Find Large / Unexpected Objects



Inspect Retainers



Find Reference Chain

39. Retainers কী?

ধরো তুমি দেখলে একটি বড় object এখনও memory-তে আছে।

প্রশ্ন:

“এটা এখনও কেন আছে?”

DevTools-এর retaining path/retainer information সাহায্য করতে পারে।

Conceptually:

```
GC Root
  ↓
Global Object
  ↓
Cache
  ↓
Large Object
```

তাহলে বুঝবে:

```
Cache
  ↓
keeps object reachable
```

এটাই leak investigation-এর মূল skill।

40. Comparison Between Snapshots

Memory leak investigate করার powerful technique:

```
Snapshot A
  ↓
Perform action
  ↓
Snapshot B
  ↓
Repeat action
```

↓
Snapshot C

ধরো:

Open modal
Close modal
Take snapshot

তারপর আবার:

Open modal
Close modal
Take snapshot

যদি প্রতিবার modal-related objects retained হতে থাকে:

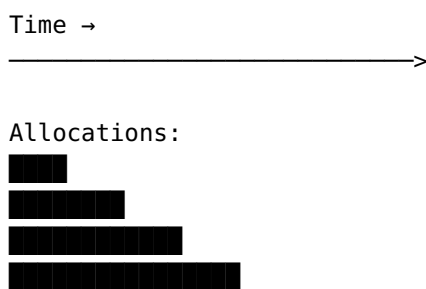
Snapshot 1 → 100 objects
Snapshot 2 → 200 objects
Snapshot 3 → 300 objects

তাহলে leak-এর strong signal পাওয়া যায়।

41. Allocation Timeline / Allocation Instrumentation

Memory সময়ের সাথে কীভাবে allocate হচ্ছে সেটা দেখতে allocation profiling useful।

Mental model:



এতে বোঝা যায়:

- কখন objects allocate হচ্ছে
- কোন operation-এর সময় allocation বাড়ছে
- কোন objects পরে release হচ্ছে না

42. Allocation Timeline কেন Useful?

ধরো একটি button click করলে:

```
Click
↓
Open modal
↓
Allocate objects
↓
Close modal
↓
Expected cleanup
```

কিন্তু memory:

```
Click 1 → +10 MB
Click 2 → +10 MB
Click 3 → +10 MB
Click 4 → +10 MB
```

এটা investigate করার strong signal ।

43. Heap Snapshot বনাম Allocation Timeline

Tool	Best for
Heap Snapshot	কোন objects memory-তে আছে
Snapshot Comparison	কোন objects retained হচ্ছে
Allocation Timeline/Instrumentation	কখন কী allocate হচ্ছে
Allocation Sampling	Allocation hotspots বোঝা

44. Practical Memory Leak Investigation

ধরো application-এ:

“Modal বারবার open/close করলে memory বাড়ে।”

Investigation:

Step 1

DevTools → Memory

Step 2

Initial Heap Snapshot নাও।

Step 3

Modal open/close করো কয়েকবার।

Step 4

আরেকটি snapshot নাও।

Step 5

Compare করো।

Step 6

Modal-related objects খুঁজো।

Step 7

Retaining path দেখো।

Step 8

Reference কোথা থেকে আসছে identify করো।

Step 9

Cleanup ঠিক করো।

Step 10

আবার profile করো।

45. Example: Event Listener Leak Investigation

ধরো:

```
function createWidget() {  
  const data = new Array(100000).fill("x");  
  
  const handler = () => {  
    console.log(data.length);  
  };  
  
  window.addEventListener("resize", handler);  
}
```

প্রতিবার:

```
createWidget();
```

call করলে নতুন listener + closure তৈরি হচ্ছে।

```
Window  
↓  
resize listeners  
├─ handler → data  
├─ handler → data  
├─ handler → data  
└─ handler → data
```

Cleanup না থাকলে retained memory বাড়তে পারে।

46. Better Pattern

```
function createWidget() {  
  const data = new Array(100000).fill("x");
```

```

function handler() {
  console.log(data.length);
}

window.addEventListener("resize", handler);

return function cleanup() {
  window.removeEventListener("resize", handler);
};
}

const cleanup = createWidget();

cleanup();

```

Mental model:

```

Create
  ↓
Register
  ↓
Use
  ↓
Cleanup

```

47. React Memory Management

React application-4 common cleanup areas:

- Event listeners
- `setInterval`
- `setTimeout`
- WebSocket
- subscriptions
- observers
- AbortController
- external libraries

Example:

```

useEffect(() => {
  const controller = new AbortController();

```

```
fetch("/api/users", {
  signal: controller.signal
});

return () => {
  controller.abort();
};
}, []);
```

আর:

```
useEffect(() => {
  const id = setInterval(() => {
    console.log("tick");
  }, 1000);

  return () => {
    clearInterval(id);
  };
}, []);
```

48. Memory Leak Prevention Rules

Rule 1 — Long-lived references minimize করো

যে reference-এর lifetime unnecessarily long, সেটা investigate করো।

Rule 2 — Cleanup lifecycle define করো

Create → Use → Cleanup

Rule 3 — Unlimited cache avoid করো

Cache policy রাখো।

Rule 4 — Event listeners cleanup করো

Rule 5 — Timers clear করো

Rule 6 — Subscriptions unsubscribe করো

Rule 7 — Detached DOM references avoid করো

Rule 8 — Closures-এর captured data সম্পর্কে সচেতন থাকো

49. Memory Optimization ≠ Garbage Collection Tuning

Application memory improve করার প্রথম step সাধারণত:

```
Better object lifetime  
Better data structures  
Better cache policy  
Better cleanup
```

GC tuning পরে।

Senior developer হিসেবে:

GC-কে optimize করার আগে application-এর unnecessary allocations এবং retained references খুঁজে বের করো।

50. Allocation বনাম Retention

এই দুটো আলাদা concept।

Allocation

কত memory নতুন করে তৈরি হচ্ছে।

```
new objects → allocations
```

Retention

কত memory এখনও reachable হয়ে আছে।

references → retained memory

Memory leak-এর ক্ষেত্রে retention খুব গুরুত্বপূর্ণ।

51. Memory Pressure

যদি application খুব allocation করে:

High Allocation Rate

↓

Heap grows

↓

GC runs more

↓

CPU overhead

↓

Potential latency

তাই performance খুঁজ:

“কত memory ব্যবহার হচ্ছে?”

এর ব্যাপার নয়।

এটাও দেখতে হবে:

“কত দ্রুত memory allocate হচ্ছে এবং কতক্ষণ retain হচ্ছে?”

52. Memory Leak বনাম High Memory Usage

দুটো এক জিনিস নয়।

High memory usage

Application legitimately বড় data ধরে রাখতে পারে।

Large dataset
↓
Expected memory

Memory leak

Data আর দরকার নেই, কিন্তু references থাকার কারণে retained ।

No longer needed
↓
Still reachable
↓
Memory leak

53. Important GC Mental Model

সবচেয়ে গুরুত্বপূর্ণ formula:

Reachable → Keep
Unreachable → Eligible for GC

কিন্তু:

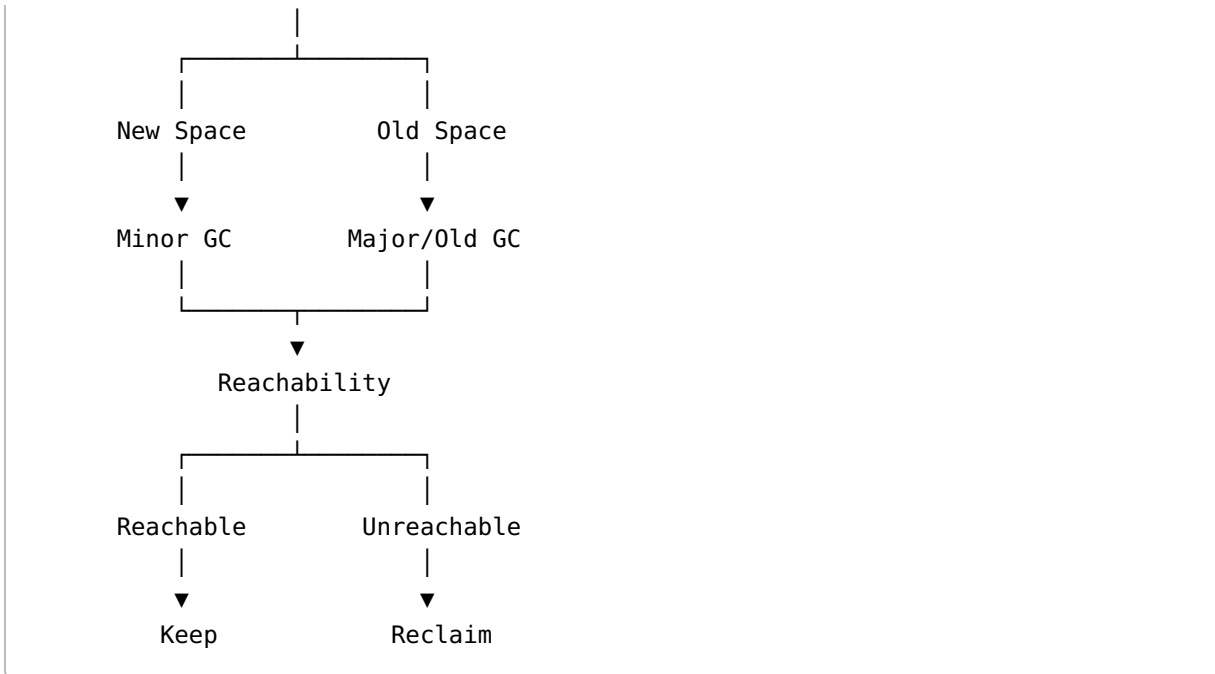
Eligible for GC মানেই **immediately memory free** হয়ে যাবে—এমন **guarantee** নেই ।

GC কখন এবং কীভাবে run করবে সেটা runtime-এর ব্যাপার ।

54. Advanced Mental Model

পুরো system:

JavaScript
↓
Allocation
↓
Heap



55. Memory Leak Mental Model

```
Object created
  ↓
Application no longer needs it
  ↓
But reference remains
  ↓
Object is still reachable
  ↓
GC cannot reclaim it
  ↓
Retained memory
  ↓
Repeated over time
  ↓
Memory leak
```

56. Senior Developer Debugging Flow

Production application slow/memory-heavy হলে:

```
Symptom
↓
Measure
↓
Chrome DevTools / Node profiling
↓
Heap Snapshot
↓
Compare snapshots
↓
Find retained objects
↓
Inspect retaining path
↓
Identify unnecessary reference
↓
Fix lifecycle / cleanup
↓
Profile again
```

Guess করে **memory leak fix** করবে না।

57. Practice Tasks

Level 1

Explain:

```
New Space
Old Space
Minor GC
Major GC
```

নিজের ভাষায়।

Level 2

এই code-এর leak identify করো:

```
const cache = [];  
  
function addData(data) {  
  cache.push(data);  
}
```

প্রশ্ন:

- কেন memory বাড়তে পারে?
- GC কেন objects reclaim করতে পারবে না?
- কী ধরনের cache policy ব্যবহার করা যায়?

Level 3

এই code analyze করো:

```
function register() {  
  const hugeData = new Array(1000000).fill("x");  
  
  window.addEventListener("click", () => {  
    console.log(hugeData.length);  
  });  
}
```

প্রশ্ন:

1. প্রতিবার `register()` call করলে কী হচ্ছে?
2. Closure কী ধরে রাখছে?
3. Listener cleanup না করলে কী সমস্যা হতে পারে?
4. কীভাবে fix করবে?

Level 4 — DevTools Challenge

একটি ছোট webpage বানাও:

```
const items = [];  
  
function createLeak() {  
  items.push(new Array(100000).fill("leak"));  
}
```

তারপর:

```
createLeak();  
createLeak();  
createLeak();
```

DevTools Memory panel দিয়ে:

1. Heap Snapshot নাও
2. আবার allocations তৈরি করো
3. দ্বিতীয় snapshot নাও
4. Compare করো
5. retained objects identify করো

58. Interview Questions

Fundamentals

1. JavaScript Heap কী?
2. Stack এবং Heap-এর conceptual difference কী?
3. New Space কী?
4. Old Space কী?
5. Generational GC কেন ব্যবহার করা হয়?
6. Minor GC কী?
7. Major GC কী?
8. Mark-and-Sweep কী?
9. Reachability কী?
10. GC Root কী?

Advanced

1. Circular reference থাকলে object কি অবশ্যই memory leak করবে?
2. Closure কি memory leak?
3. Detached DOM node কী?
4. Event listener কীভাবে memory retain করতে পারে?
5. Unbounded cache কীভাবে memory leak তৈরি করতে পারে?
6. Reachable কিন্তু unused object কেন GC করতে পারে না?
7. Minor GC এবং major GC-এর মধ্যে trade-off কী?
8. Allocation এবং retention-এর মধ্যে difference কী?
9. Heap Snapshot কী?
10. Retaining path কী?
11. দুইটি Heap Snapshot compare করার সুবিধা কী?
12. Allocation timeline কখন ব্যবহার করবে?
13. High memory usage এবং memory leak-এর মধ্যে পার্থক্য কী?

14. কেন GC automatic হলেও developer-এর memory management সম্পর্কে জানা দরকার?

59. Quick Revision Cheat Sheet

Concept	Core Idea
Heap	Dynamic runtime memory
New Space	Young-generation objects
Old Space	Long-lived objects
Generational GC	Object lifetime অনুযায়ী GC strategy
Minor GC	Young generation collection
Major GC	Older/larger heap collection
Mark	Reachable objects identify করা
Sweep	Unreachable memory reclaim করা
GC Root	Reachability-এর starting points
Memory Leak	Unnecessary memory still reachable
Detached DOM	DOM থেকে removed কিন্তু reference retained
Closure retention	Closure captured data retain করতে পারে
Heap Snapshot	Heap-এর point-in-time view
Allocation Timeline	Allocation behavior over time
Retainer	কোন reference object-টাকে alive রাখছে

60. Final Takeaways

এই পুরো section থেকে সবচেয়ে গুরুত্বপূর্ণ concepts:

1. JavaScript automatic memory management ব্যবহার করে

Developer manually সাধারণত `free()` করে না।

2. কিন্তু automatic GC মানে automatic leak prevention নয়

কারণ:

```
Still reachable
  ↓
GC cannot reclaim
```

3. New objects সাধারণত young generation-এ শুরু হয়

Long-lived objects eventually older generation-এ যেতে পারে।

4. Generational GC-এর মূল assumption

Most objects are short-lived.

5. Mark-and-Sweep reachability-এর উপর ভিত্তি করে

```
Reachable → Keep
Unreachable → Reclaim
```

6. Closure নিজে leak নয়

কিন্তু long-lived closure unnecessary data retain করতে পারে।

7. Event listeners cleanup গুরুত্বপূর্ণ

বিশেষ করে manually registered listeners-এর ক্ষেত্রে।

8. Detached DOM nodes investigate করতে হয়

DOM থেকে remove হওয়া মানেই memory থেকে remove হওয়া নয়।

9. Memory leak diagnose করতে profiling ব্যবহার করো

Guess নয়:

```
Measure
  ↓
Snapshot
  ↓
Compare
  ↓
Retaining Path
  ↓
Fix
```



Measure Again

10. Senior-level memory management-এর মূল প্রশ্ন

কখনো শুধু:

“কত memory ব্যবহার হচ্ছে?”

এটা জিজ্ঞেস করবে না।

বরং জিজ্ঞেস করবে:

“কোন **objects memory**-তে আছে, কেন তারা এখনও **reachable**, কতক্ষণ ধরে আছে, এবং তাদের **lifetime** কি সত্যিই প্রয়োজনীয়?”

এটাই JavaScript memory management-এর সবচেয়ে গুরুত্বপূর্ণ mental model।